

## Lab 2: LinkedList Operation Implementation

**Due: 11:59pm** on Mar 25, 2025 (Tuesday)

Whenever there is a question in relation to LinkedList, you should first ask whether it is a singly linked list or a doubly linked list because your algorithm can be implemented quite differently.

Many problems with a singly linked list can be solved using “runner” (or second pointer) technique.

What does that mean? It means you iterate through a linked list with two pointers (references) at the same time. In most cases, you put one pointer ahead of the other.

Let's call the pointer that is in front of the other, “front.”  
And, we can call the other pointer, “back.”

For example:

We have a singly linked list in a way that  $D1 \rightarrow D2 \rightarrow \dots \rightarrow D_{n-1} \rightarrow D_n \rightarrow S1 \rightarrow S2 \rightarrow \dots \rightarrow S_{n-1} \rightarrow S_n$   
What we want to have as the result of your algorithm is to restructure the list to be  $D1 \rightarrow S1 \rightarrow D2 \rightarrow S2 \rightarrow \dots \rightarrow D_n \rightarrow S_n$ . We have no information about the length of the list but we do know that the number of nodes are even.

How would we do it?

We could have two pointers (front and back). Make front move two steps (*it runs*) while back moves one step (*it walks*). As front, or the runner, hits the end of the list, back, or the walker, must be at the midpoint. Then have the front pointer to (re-)point to the head of the list. We can now weave the head of the list and the node at the midpoint in each iteration.

Now, let's solve another problem.

**Problem:**

You are to write a method to find kth node to the last element of a singly linked list. The method should return null if k is less than 1 or bigger than the length of the list. *We do NOT know the length of the list in advance, and want to return an answer in at most one pass through the linked list.*

Download the Lab 2 file (LinkedListLab.java) from Canvas and try to implement the kthToLast() method of the LinkedListLab class.

You may want to create a lab2 Java project using your choice of IDE, and put the LinkedListLab.java file into the project.

1. What is the return value for the k of 2?
2. What is the return value for the k of 4?
3. Using Big O notation, this algorithm's time complexity is \_\_\_\_\_.

Submit your LinkedListLab.java file using Autolab (<https://autolab.andrew.cmu.edu>). Do not zip it the file. Make sure to submit your source code file (.java file), not .class file.

The questions in this handout are for you to think about. Please do not submit this handout.